



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure (P)
Environment (E)
Actuators (A)
Sensors (S)

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.grid_width, self.grid_height – grid dimensions
self.walls – obstacle positions
self.food – food locations
self.agent_pos – agent starting position
self.opponents – opponent positions

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent environment (justified by `self.opponents` presence)
Multiple agents with potentially conflicting goals affect the shared environment

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the ordered history of all perceptions (sensory inputs) an agent receives over time. Each percept is a snapshot of the environment state at a given time step

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

`get_percept()` represents the Sensors (S) component
It translates raw environment state into observable data for the agent

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable environment

The agent can only see what's in the dictionary returned by `get_percept()` (likely local vicinity, food, etc.)

The agent cannot see: all walls, all food positions, opponent long-term strategies

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

`execute_action()` updating the Performance Measure (P)
Score deduction reflects external environment consequences

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Rationality is defined by outcomes, not effort
Internal processing effort is irrelevant, only environmental impact affects agent performance
Prevents agents from being "rewarded" for thinking without acting effectively

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Environment becomes more hidden , increased partial observability
 Agent operates under greater uncertainty
 The environment has features the agent cannot directly perceive

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'` , you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Provides utility-relevant information for decision-making
 Agent can now choose actions that maximize expected utility by avoiding traps
 Rational agents need percepts tied to performance metrics
 Better information , better decisions , higher expected performance

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

Problem: If a vacuum agent's score rewarded "dirt detection" rather than "dirt removal," the agent could exploit this by:

Constantly moving without cleaning

Or repeatedly detecting the same dirt

Maximizing score without achieving actual task performance

Lesson: Performance measures must directly reflect external outcomes, not intermediate observations

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING